

**МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ**  
**НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА**



**АВТОМАТИЗОВАНЕ**  
**ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ**  
**СИСТЕМ**

Лабораторна робота №2

“Створення простої комунікації SW(client) <-> UART <-> HW(server)”

**Виконав:**  
студент гр. КІ-404  
Гіряк О.Р.  
**Прийняв:**  
Федак П. Р.

### Завдання:

1. Create a simple communication schema SW(client) <-> UART <-> HW(server).
2. The client should send a message to the server. The server should modify the message and send it back to the client.
3. Create YML file with next features: a. build all binaries (create scripts in folder ci/ if need); b. run tests; c. create artifacts with binaries and test reports;
4. Required steps.

### Теорія:

#### Схема комунікації SW(client) <-> UART <-> HW(server)

UART (Universal Asynchronous Receiver-Transmitter) — це протокол для асинхронної серійної передачі даних між клієнтом (програмне забезпечення) та сервером (апаратне забезпечення). Він використовує дві лінії для передачі даних: TX (передача) і RX (прийом). У цьому контексті клієнт надсилає дані через UART на сервер, який обробляє їх і відповідає назад через той самий інтерфейс.

#### Передача повідомлення від клієнта до сервера та його модифікація

Клієнт надсилає повідомлення через UART. Сервер приймає це повідомлення, модифікує його (наприклад, додає префікс або змінює частину тексту) та відправляє відповідь назад клієнту. Ключові етапи:

- Передача даних клієнтом
- Прийом повідомлення сервером
- Обробка/модифікація повідомлення
- Відповідь сервера клієнту

#### YML-файл для збірки та тестування

YML — формат для конфігураційних файлів. Для автоматизації розробки можна створити YML-файл з наступними характеристиками:

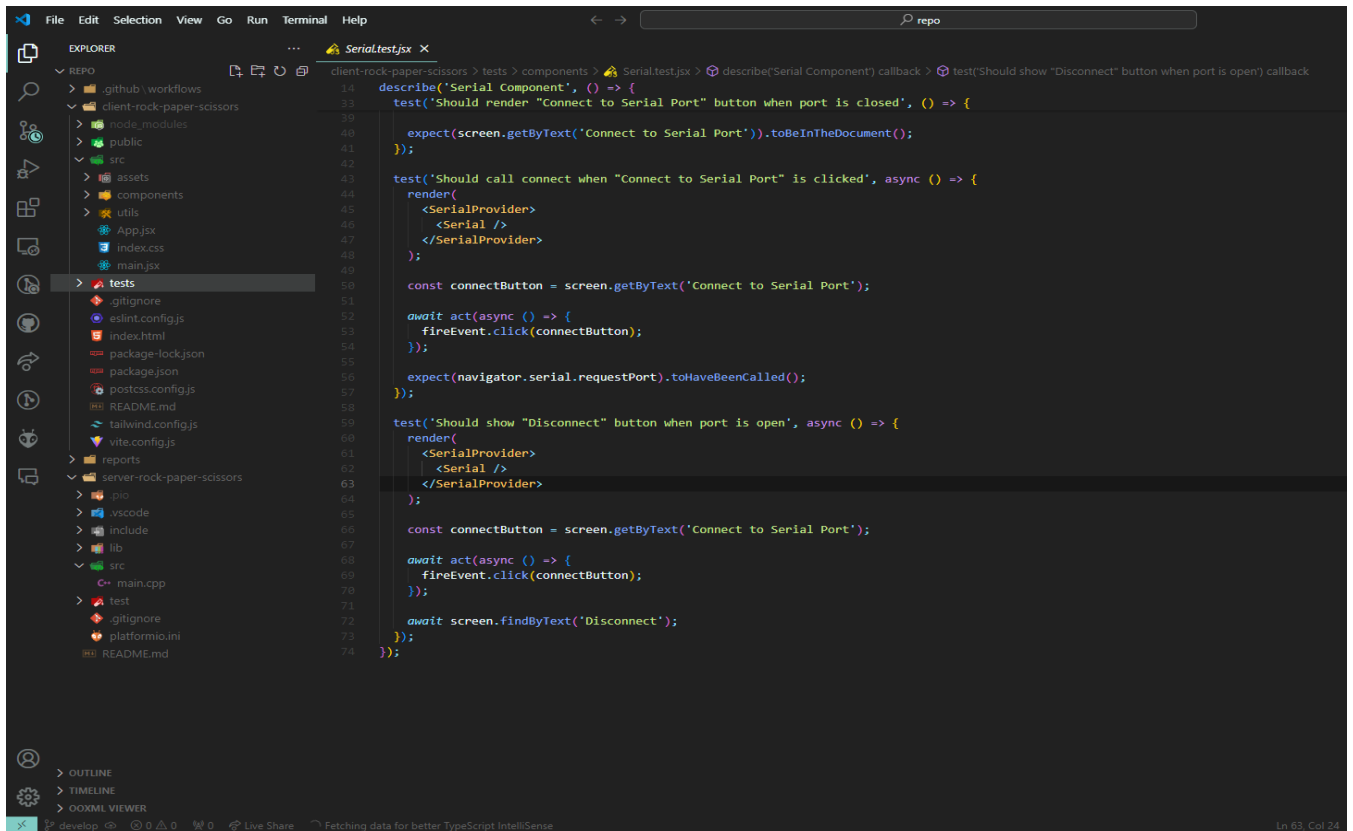
- Збірка всіх бінарних файлів (виконання скриптів із директорії ci/).
- Запуск тестів після збірки.
- Створення артефактів (бінарних файлів та звітів про тести) після виконання всіх етапів.

### Індивідуальне завдання:

Згідно списку групи я маю 2 варіант, так як я маю такий порядковий номер у списку.

| Student number | Game                | config format |
|----------------|---------------------|---------------|
| 2              | rock paper scissors | JSON          |

## Виконання:



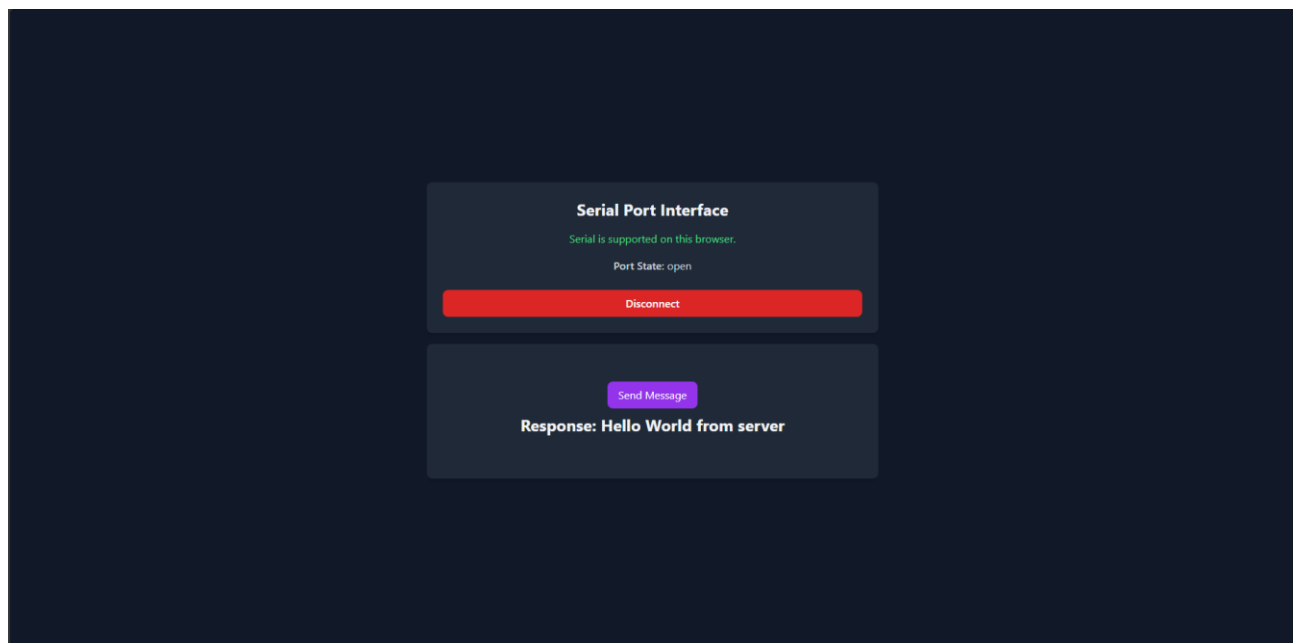
*Рис.1.* Оновлена структура проекту

На даному скріншоті продемонстровано оновлена структура проекту, були добавлені папки для клієнта і сервера, а також папка, яка містить ci.yml для GitHub Actions.

```
github: 3 workflows 2 1 ci.yml
Anton Ponomarev, 22 hours ago [1] author [Anton Ponomarev]
name: CI React Vite and PlatformIO
on:
  push:
    branches:
      - '*'
  pull_request:
    branches:
      - '*'
  workflow_dispatch:
jobs:
  # Job for building and testing the project
  client-build-and-test:
    runs-on: ubuntu-latest
    steps:
      # Step 1: Checkout the repository
      - name: Checkout repository
        uses: actions/checkout@v4
      # Step 2: Set up Node.js environment (Node.js 20)
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
      # Step 3: Install dependencies for the React Vite app
      - name: Install dependencies
        run: npm install
        working-directory: client-rock-paper-scissors
      # Step 4: Run Jest tests
      - name: Run Jest tests
        run: npm run test
        working-directory: client-rock-paper-scissors
      # Step 5: Run vite build
      - name: Build Vite project
        run: npm run build
        working-directory: client-rock-paper-scissors
      # Step 6: Upload build artifacts
      - name: Upload Vite build artifact
        uses: actions/upload-artifact@v4
        with:
          name: client-build
```

*Рис.2.* Файл ci.yml

Файл з написаним flow для роботи тестів і компіляції проектів з подальшим виведенням скомпільованих файлів у артефакти через GitHub Actions



*Рис.3.* Уі клієнту

### **Висновок:**

Виконуючи дану лабораторну роботу я створив клієнт і сервер та налаштував комунікацію між ними використовуючи серіал порт. Також, я налаштував `ci.yml` для роботи з GitHub Actions в якому прописаний `flow` для запуску тестів і білду проектів. Останніми кроками, які я зробив стало додавання тегу та створення `pull request`, який після апруву змерджив.